

UNIT 2: VIRTUALIZATION AND CONTAINERIZATION

Course Pattern: TE IT - 2024 Pattern (SPPU)
Status: Publication-Grade Theory Guide

Academic Term: Semester V (2026)
Course Code: PEC-321A-IT

Syllabus Mapping & Unit Structure

The following table outlines the exam-oriented curriculum mapping and priority weightage for Unit 2.

Syllabus Topic	In-Depth Exam Coverage	Weightage / Priority
Introduction & Benefits	Virtualization definition, physical hardware abstraction, server consolidation, isolation, live migration, green computing.	High Priority (6-8 Marks)
Types of Virtualization	Full Virtualization (binary translation), Para-Virtualization (hypercalls), OS-level, Network (SDN), Storage (SDS), Desktop (VDI).	Critical Priority (8-10 Marks)
Hypervisors (Type-I & Type-II)	Bare-metal Type-I vs Hosted Type-II architecture, Ring privilege model, hardware-assisted virtualization (VT-x/AMD-V).	Critical Priority (8-10 Marks)
Virtual Machines & Lifecycle	Anatomy of VMs (vCPU, vRAM, vDisk), VMM control, VM lifecycle (Creation, Live Migration pre/post-copy, Snapshots).	High Priority (8-10 Marks)
Containers & OS Primitives	Container concept, Linux Kernel namespaces (PID, NET, MNT, IPC, UTS, USER), Control Groups (cgroups), Copy-on-Write (CoW).	Critical Priority (8-10 Marks)
Docker Architecture & Engine	Docker daemon, REST API, Dockerfile syntax, Images vs Containers, Docker Hub, Volumes, and Networking modes.	Critical Priority (8-10 Marks)
VMs vs Containers Matrix	Side-by-side technical comparison across isolation, startup time, footprint, performance overhead, kernel dependency.	High Priority (6-8 Marks)
Deployment & Kubernetes	Docker commands (build, run, ps), Docker Compose, Kubernetes Master/Worker architecture, Pods, Services, Deployments.	Critical Priority (8-10 Marks)

Introduction to Virtualization & Core Concepts

Definition of Virtualization

Virtualization is a fundamental technology in cloud computing that creates an abstract, software-based representation of physical computing resources—including CPU, memory, storage arrays, network switches, and graphics processing units.

By placing a software abstraction layer (the **Hypervisor** or **Virtual Machine Monitor / VMM**) directly above the physical hardware, virtualization decouples physical hardware resources from the operating systems and application software running upon them.

NOTE:

Virtualization transforms fixed, single-purpose physical machines into dynamic, flexible pools of logical resources that can be partitioned, allocated, and scaled dynamically among multiple independent virtual machines (VMs) or execution environments.

Role of Virtualization in Cloud Infrastructure

Virtualization serves as the foundational enabler of modern cloud computing architectures by providing:

1. **Multi-Tenancy:** Allows different customers (tenants) to share the same physical server securely without exposing sensitive execution state or data across tenant boundaries.
2. **Hardware Abstraction:** Standardizes heterogeneous hardware components (Intel, AMD CPUs, NVMe, SATA drives, Mellanox NICs) into uniform, virtualized interfaces for guest operating systems.
3. **Elasticity & Dynamic Resource Allocation:** Allows cloud control planes to dynamically adjust memory limits, CPU shares, and disk capacities without physical hardware modification or extended downtime.

Source: Srinivasan & Suresh, Ch 8, p. 100-105 | Source: Buyya / Erl, Ch 3, p. 71-75

Benefits & Applications of Virtualization

Key Operational & Economic Benefits

1. **Server Consolidation:**
 - Traditional physical servers often run at low average CPU utilization (5% - 15%).
 - Virtualization enables packing dozens of virtual servers onto a single high-density physical node, raising average hardware utilization to 70% - 80%.
2. **Hardware Independence & Portability:**
 - Virtual machines run inside encapsulated software files (e.g., .vhdx, .vmdk, .qcow2).
 - A VM configured on an Intel Xeon host can easily be migrated and executed on different server hardware without reinstalling operating systems or device drivers.
3. **Strong Workload Isolation & Fault Containment:**
 - A kernel panic, security breach, or memory crash occurring inside one Guest OS is strictly isolated within its virtual partition, preventing damage to other co-located VMs or the host physical node.
4. **Rapid Provisioning & Automated Cloning:**
 - Spinning up a new physical server requires physical procurement, rack assembly, cable routing, OS installation, and configuration (taking days or weeks).
 - Spinning up a virtual machine from a pre-configured template image takes seconds or minutes.
5. **Disaster Recovery & Snapshots:**
 - Administrators can capture live point-in-time snapshots of an entire virtual machine's RAM and disk state, enabling instant rollback in case of software failure or ransomware infection.

6. Green Cloud Computing & Energy Reduction:

- Reducing physical server count directly scales down server power consumption, uninterruptible power supply (UPS) footprints, and data center HVAC cooling requirements.

Source: Srinivasan & Suresh, Ch 8, p. 116-120 | Source: Buyya / Erl, Ch 3, p. 85-90

Types & Taxonomy of Virtualization

Virtualization techniques span multiple architectural layers of a computing environment:

Full Virtualization

- Mechanism:** The guest operating system is completely unaware that it is running in a virtualized environment. It executes unmodified OS kernel binaries.
- Execution:** The hypervisor intercepts non-trappable privileged CPU instructions using **Binary Translation** or leverages hardware CPU extensions (Intel VT-x, AMD-V) to execute sensitive instructions in a protected guest mode (Ring 1 or VMX non-root mode).
- Pros:** High compatibility; any off-the-shelf OS (Windows, Linux, BSD) can be installed without source code modification.
- Cons:** CPU instruction translation adds minor performance overhead.
- Key Implementations:** VMware Workstation, QEMU, VirtualBox (without guest additions).

Para-Virtualization

- Mechanism:** The guest operating system kernel is explicitly modified to be aware of the underlying virtualization layer.
- Execution:** Privileged hardware calls (such as page table manipulations, timer resets, and I/O access) are replaced with direct software API calls called **Hypercalls** routed directly to the hypervisor.
- Pros:** Near-native CPU and I/O execution speeds; eliminates complex binary translation overhead.
- Cons:** Requires access to guest OS source code; cannot run proprietary unmodified operating systems directly.
- Key Implementations:** Xen (Classic PV mode), KVM (using `virtio` paravirtualized device drivers).

Hardware-Assisted Virtualization

- Mechanism:** Modern CPU architectures (Intel VT-x / AMD-V) provide built-in hardware extensions including a dedicated execution mode (VMX Root Operation for Hypervisor, VMX Non-Root Operation for Guest OS) and hardware page table translation via Extended Page Tables (EPT) / Nested Page Tables (NPT).
- Pros:** Eliminates software binary translation overhead while running unmodified guest operating systems at near-bare-metal speed.

Domain-Specific Virtualization Types

Virtualization Domain	Architectural Mechanism	Primary Examples
Server Virtualization	Partitioning physical server hardware into multiple isolated virtual machines managed by a hypervisor.	VMware ESXi, KVM, Microsoft Hyper-V.
Network Virtualization (SDN)	Decouples network control plane from physical data forwarding plane, creating software-defined virtual switches, routers, and VLANs.	VMware NSX, Open vSwitch (OVS), Cisco ACI.
Storage Virtualization (SDS)	Pools multiple physical storage arrays (SAN/NAS/Disks) into unified virtual storage pools abstraction.	AWS EBS, Ceph, VMware vSAN.
Desktop Virtualization (VDI)	Runs desktop operating systems inside centralized data center VMs and streams user display output to thin client devices.	Citrix Virtual Apps & Desktops, VMware Horizon.

Virtualization Domain		Architectural Mechanism	Primary Examples
Application	Virtualization	Encapsulates application dependencies and registry keys into isolated sandboxes executing above host OS without full OS installation.	Microsoft App-V, VMware ThinApp.

Source: Srinivasan & Suresh, Ch 8, p. 105-115 | Source: Buyya / Erl, Ch 3, p. 76-84

Hypervisor Architecture (Type-I vs Type-II)

The Hypervisor (or Virtual Machine Monitor - VMM) is the specialized software component responsible for managing physical resource allocation, scheduling virtual CPUs, intercepting privileged instructions, and maintaining strict isolation between virtual machines.

Type-I Bare-Metal Hypervisors

- **Architecture:** Type-I hypervisors run directly on top of bare-metal physical hardware without requiring a host operating system.
- **Execution Model:** The hypervisor contains its own lightweight microkernel, device drivers, and CPU scheduler. It boots as the primary operating layer on physical hardware startup.
- **Performance:** Minimal overhead, low latency, extremely high I/O throughput.
- **Enterprise Deployment:** Used exclusively in commercial cloud data centers (AWS, Azure, GCP) and enterprise server farms.
- **Key Implementations:** VMware ESXi, KVM (Kernel-based Virtual Machine built into Linux kernel), Xen, Microsoft Hyper-V.

Type-II Hosted Hypervisors

- **Architecture:** Type-II hypervisors run as application software on top of an existing host operating system (e.g., Windows, macOS, Linux).
- **Execution Model:** The host OS manages physical hardware drivers and memory scheduling. The hypervisor relies on the host OS to issue hardware instructions, creating an extra software layer between the VM and physical hardware.
- **Performance:** Higher execution overhead due to double scheduling (Guest OS -> Hypervisor -> Host OS -> Hardware).
- **Deployment:** Software development, local testing, personal desktop environments.
- **Key Implementations:** Oracle VirtualBox, VMware Workstation / Fusion, Parallels Desktop.

Comprehensive Comparison Matrix: Type-I vs. Type-II

Comparison Parameter	Type-I (Bare-Metal / Native)	Type-II (Hosted)
Installation Layer	Directly on bare-metal physical hardware.	On top of a general-purpose host OS.
Operating System Dependency	None (Contains embedded microkernel/drivers).	Strictly dependent on host OS stability.
Resource Overhead	Extremely low (< 2% - 5%).	Higher due to host OS footprint (15% - 25%).
Execution Speed & IOPS	Near-native CPU, RAM, and Disk throughput.	Slower due to host OS system call translation.
Security Isolation	Highest; single-purpose hardened codebase.	Vulnerable to compromise if host OS is breached.
Typical Market Deployment	Hyperscale clouds (AWS, Azure) & enterprise datacenters.	Developer workstations, local software testing.

Comparison Parameter	Type-I (Bare-Metal / Native)	Type-II (Hosted)
Leading Commercial Vendors	VMware ESXi, KVM, Xen, Microsoft Hyper-V.	Oracle VirtualBox, VMware Workstation.

Source: Sosinsky, Ch 5, p. 130-145 | Source: Srinivasan & Suresh, Ch 8, p. 114-116

Virtual Machine Architecture & Lifecycle Management

Virtual Machine Anatomy

A Virtual Machine (VM) is a software-based emulation of a physical computer system consisting of:

- **Virtual CPU (vCPU):** Represented as execution threads scheduled by the hypervisor onto underlying physical CPU cores.
- **Virtual Memory (vRAM):** Guest OS physical memory mapped by hypervisor page tables to physical host RAM.
- **Virtual Disk (vDisk):** Abstracted file containers (.vmdk, .qcow2, .vhdx) presenting raw block device structures to guest storage drivers.
- **Virtual Network Interfaces (vNICs):** Software network adapters assigned virtual MAC addresses connected to virtual switches.

Complete Virtual Machine Lifecycle

1. **Creation & Provisioning:**
 - Hypervisor allocates hardware resource caps (vCPU count, RAM capacity, vDisk size).
 - Guest OS is installed from an ISO file or instantiated instantly from a golden template image.
2. **Execution & State Monitoring:**
 - Hypervisor intercepts interrupts and traps privileged operations.
 - Resource utilization (CPU usage, memory pressure, I/O rates) is continuously monitored.
3. **Live Migration (Hot Migration):**
 - Moving an actively running virtual machine from one physical server node to another without noticeable downtime or service interruption to connected end-users.
 - **Pre-Copy Live Migration Algorithm:**
 1. Hypervisor copies all memory pages from Source Host to Destination Host while VM continues running.
 2. Memory pages modified during transfer ("dirty pages") are tracked in a bitmap.
 3. Iterative rounds copy dirty pages until dirty page generation rate stabilizes.
 4. Source VM is briefly paused (milliseconds), final CPU state and dirty pages are transferred, and execution resumes on Destination Host.
 - **Post-Copy Live Migration Algorithm:** Source VM suspends immediately, transfers minimal CPU execution state to Destination Host, and resumes execution; missing memory pages are fetched on-demand via network page faults.
4. **Snapshotting & Rollback:**
 - Preserves state, memory contents, and disk image blocks at an exact timestamp for instant system state restoration.
5. **Decommissioning & Teardown:**
 - Reclaims host RAM, deletes vDisk files, and frees virtual switch port bindings.

Source: Buyya / Erl, Ch 3, p. 80-88 | Source: Sosinsky, Ch 5, p. 135-140

Containers & OS-Level Virtualization

The Container Concept

Containerization (OS-Level Virtualization) is an architectural approach where multiple isolated user-space environments—called **Containers**—run directly on top of a single shared Linux operating system kernel.

Unlike Virtual Machines, containers do **not** run separate guest operating systems or hypervisors. Instead, containers bundle application binaries, code libraries, dependencies, and configuration files into lightweight execution units that share the host kernel.

IMPORTANT:

Containers virtualize the **operating system kernel**, whereas Virtual Machines virtualize the **physical server hardware**. This fundamental distinction gives containers extreme portability, minimal memory footprints, and sub-second startup times.

Linux Kernel Primitives Enabling Containerization

Container engines (such as Docker and containerd) rely on three underlying core Linux kernel technologies:

1. **Linux Namespaces (Process Isolation):** Namespaces provide isolated views of system resources for each container process tree:
 - `pid` (Process ID): Isolates process trees; container process sees itself as PID 1.
 - `net` (Networking): Provides isolated network interfaces, IP addresses, routing tables, and firewall rules.
 - `mnt` (Mount): Isolates file system mount points.
 - `ipc` (Inter-Process Communication): Isolates System V IPC and POSIX message queues.
 - `uts` (UNIX Timesharing System): Allows containers to define custom hostnames and domain names.
 - `user` (User IDs): Maps container root users (UID 0) to unprivileged UIDs on host.
2. **Control Groups (cgroups) (Resource Management):**
 - `cgroups` limit, account for, and isolate physical resource usage (CPU cores, RAM memory caps, disk I/O bandwidth, network bandwidth) among container process groups.
 - Prevents a single misbehaving container from exhausting system resources (denial-of-service).
3. **Copy-on-Write (CoW) Union File Systems:**
 - OverlayFS and AUFS combine multiple underlying directory layers into a single unified file system view.
 - Read-only base image layers are shared concurrently among hundreds of containers; modifications are written exclusively to a thin container-specific read-write top layer.

Source: Srinivasan & Suresh, Ch 8, p. 108-112 | Source: Buyya / Erl, Ch 3, p. 90-95

Docker Fundamentals & Architecture

Docker Engine Client-Server Architecture

Docker is an open-source containerization platform structured around a client-server architecture:

1. **Docker CLI (docker command-line client):** User interface for issuing commands (`docker run`, `docker build`, `docker pull`).
2. **Docker Daemon (dockerd):** Persistent background process managing Docker objects (containers, images, networks, volumes) via REST API calls.
3. **Container Runtime (containerd & runc):** Low-level OCI (Open Container Initiative) compliant execution runtime that directly interacts with Linux kernel namespaces and `cgroups`.

Docker Images vs. Containers

- **Docker Image:** An immutable, read-only template built from a sequence of stacked read-only file system layers. Images contain application code, runtime libraries, environment variables, and default entrypoint commands.
- **Docker Container:** A runnable instance of a Docker Image. Instantiating a container adds a thin writable layer (container layer) on top of the immutable image layers.

Dockerfile Syntax & Instructions

A Dockerfile is a text script containing consecutive instruction steps used to automatically build a Docker Image:

```
# Step 1: Specify base image layer
FROM node:18-alpine

# Step 2: Set working directory inside container
WORKDIR /app

# Step 3: Copy dependency manifests and install packages
COPY package*.json ./
RUN npm install --production

# Step 4: Copy application source code
COPY . .

# Step 5: Expose application network port
EXPOSE 3000

# Step 6: Set environment variables
ENV NODE_ENV=production

# Step 7: Specify default execution command on container start
CMD ["node", "server.js"]
```

Docker Storage & Networking Modes

- **Docker Storage Drivers:** OverlayFS2 manages copy-on-write image layers.
- **Docker Volumes:** Persistent data storage volumes managed by Docker outside the container writable layer (/var/lib/docker/volumes), surviving container deletion.
- **Docker Networking Modes:**
 - bridge (Default): Creates a virtual bridge interface (docker0); containers receive private internal IP addresses mapped to host ports via NAT.
 - host: Removes network isolation; container shares host network interface directly.
 - none: Disables networking completely for isolated security processing.
 - overlay: Enables multi-host container networking across Docker Swarm nodes.

Source: Sosinsky, Ch 5, p. 140-148 | Source: Srinivasan & Suresh, Ch 10, p. 130-135

Deep Technical Comparison: VMs vs. Containers

Architectural Dimension	Virtual Machines (VMs)	Containers (e.g., Docker)
Abstraction Layer	Hardware-level virtualization via Hypervisor.	OS-level virtualization via Linux kernel primitives.
Guest Kernel	Each VM runs an independent Guest OS kernel.	All containers share host OS kernel.
Startup Time	Minutes (Full OS boot cycle required).	Seconds or Milliseconds (Process start).

Architectural Dimension	Virtual Machines (VMs)	Containers (e.g., Docker)
Storage Footprint	Large (Gigabytes; includes guest OS binaries).	Small (Megabytes; shares base image layers).
Memory Overhead	High (Pre-allocated RAM assigned to VM).	Extremely low (Dynamic RAM per process).
Security Isolation	Strong hardware-enforced isolation (Hypervisor boundary).	Process isolation via namespaces (Shared kernel surface).
Portability	High within same hypervisor format.	Maximum; runs anywhere Docker Engine is installed.
Deployment Density	Dozens of VMs per physical host server.	Hundreds/Thousands of containers per host server.

Source: Srinivasan & Suresh, Ch 8, p. 108-110 | Source: Sosinsky, Ch 5, p. 142-146

Basic Container Deployment & Introduction to Kubernetes

Multi-Container Orchestration with Docker Compose

Docker Compose allows defining and running multi-container applications using a declarative YAML configuration file (docker-compose.yml):

```
version: '3.8'
services:
  web-app:
    build: .
    ports:
      - "8080:3000"
    environment:
      - DB_HOST=db
    depends_on:
      - db
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_PASSWORD: secretpassword
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Introduction to Kubernetes (K8s) Architecture

Kubernetes is an open-source production-grade container orchestration system designed to automate deployment, scaling, load balancing, and management of containerized applications across cluster nodes.

Kubernetes Master Control Plane Architecture

1. **kube-apiserver**: Central REST gateway exposing the Kubernetes API and validating cluster configuration requests.
2. **etcd**: Highly available, distributed key-value storage maintaining cluster state data.
3. **kube-scheduler**: Assigns newly created Pods to optimal Worker Nodes based on resource availability and constraints.
4. **kube-controller-manager**: Runs controller loops managing cluster state (NodeController, ReplicaSetController).

Kubernetes Worker Node Components

1. **kubelet**: Primary node agent ensuring assigned container Pods are running and healthy.
2. **kube-proxy**: Network proxy managing cluster virtual network routing and TCP/UDP load balancing.
3. **Container Runtime**: Underneath software (e.g., containerd, CRI-O) executing containers inside Pods.

Core Kubernetes Abstraction Objects

- **Pod**: The smallest deployable computing unit in Kubernetes, wrapping one or more tightly coupled containers sharing network IP and storage volumes.
- **Service**: An abstract path exposing a set of Pods as a network service with a persistent Virtual IP (ClusterIP, NodePort, LoadBalancer).
- **Deployment**: Declarative specification defining desired Pod state, replica counts, rolling updates, and automated rollbacks.

Source: Buyya / Erl, Ch 3, p. 95-105 | Source: Sosinsky, Ch 5, p. 145-150